ARI3217 - Reflective Diary Entry 1 Workflow

Overall approach

- Create a separate project, such as ~/Desktop/ContractReview; keep shop-model as a reference.
- Work inside Ubuntu, using SSH from your Mac for editing and commands.
- Package the supplied model and preprocessing; don't replace or tune the learner.
- Commit working changes regularly. Record measurements, decisions, failures, and screenshots as you work.
- Use your own results in the diary. The notebook's numbers are comparison targets, not your evidence. 29_contract_review.ipynb

Before Stage 1 — Establish your baseline

- Confirm you claimed project 29 on both the allocation sheet and VLE forum.
- Create your repository, virtual environment, .gitignore, and an evidence folder.
- Run the original notebook from a clean kernel.
- Record runtime, package versions, dataset counts, split sizes, and headline metrics.
- Compare against the supplied targets:
    - 297 training, 99 validation, and 99 test contracts; 15 franchise agreements held out separately.
    - Macro AUPR approximately 0.489, or 0.499 with the stated knowledge-base retriever.
    - Red-flag recall at 20 highlights: 88.0% on regular test contracts and 53.3% on held-out franchise agreements.
- Deliverable: a baseline-results table linked to a Git commit.

Stage 1 — Extract reusable functions

- Move the notebook's data loading, masking, chunking, feature construction, training, calibration, and review-pack functions into Python functions.
- Preserve the supplied processing order and configuration.
- Keep the notebook as a thin client that imports your code.
- Deliverable: a script that reproduces the baseline outside the notebook.

Stage 2 — Build an installable package

- Use src/contract_review/ with modules for data, chunking, features, models, evaluation, and review packs.

- Keep the supplied taxonomy/retrieval components needed to reproduce the baseline.
- Add pyproject.toml and a training command, such as contract-train.
- Deliverable: an editable installation that imports successfully from another directory.

Stage 3 — Separate configuration

- Move seeds, paths, chunk settings, supplied model parameters, and review budget into configuration.
- Use .env for local settings and secrets; provide env.example.
- Deliverable: configurable training without editing application code.

Stage 4 — Add logging

- Log dataset and split counts, training duration, evaluation results, and saved-artifact locations.
- Avoid logging complete contracts or unnecessary personal information.
- Deliverable: readable logs showing what each run did.

Stage 5 — Create a reliable command

- Make the training command execute the full workflow.
- Save the model, vocabulary, calibrators, and relevant taxonomy/configuration versions together.
- Return exit code 0 on success and a non-zero code on failure.
- Deliverable: one documented command to train and evaluate the system.

Stage 6 — Check data and write tests

- Verify at least three documented quirks or traps, with counts.
- Investigate one additional issue not listed; report honestly if your candidate issue turns out not to be a problem.
- Useful checks include contract-family overlap across splits, clauses cut by chunk boundaries, rare-category coverage, and annotation inconsistencies.
- Turn at least one finding into a failing test or input check; add meaningful tests for preprocessing and output citations.
- Deliverables: passing tests and a data-check table containing:
    - What the notebook says.
    - What you measured.
    - Why it matters.
    - Which test or check guards it.

Stage 7 — Make the project reproducible

- Control randomness and pin dependencies.
- Document installation, training, and evaluation.
- Run a fresh-copy test with a new environment and no existing model artifacts.
- Reproduce the headline metrics within a stated tolerance.
- Preserve the notebook's prescribed grouped split. The diary also requests forward-time evaluation: document this difference and clarify the expected additional evaluation with your lecturer rather than silently replacing the supplied split.
- Deliverable: original-versus-fresh-copy results and reproducible instructions.

Stage 8 — Serve contract reviews

- Add POST /review: contract text in; cited highlights and "not found" information out.
- Load saved artifacts once at startup.
- Reuse the exact training preprocessing.
- Add input validation and /health.
- Deliverable: a successful API example and a test demonstrating training–serving consistency.
- An LLM summary is not needed for this first diary entry.

Stage 9 — Containerise the service

- Create Dockerfile and .dockerignore.
- Build and run the API image.
- Check it reproduces the expected review behaviour.
- Measure and record the image size.
- Deliverables: working image, build/run instructions, image identifier, and size measurement.

Stage 10 — Store review records

- Add PostgreSQL and Docker Compose.
- Store a review/request identifier, timestamp, model version, selected-highlight metadata, and relevant scores.
- Decide whether text needs storing; prefer references or minimised records where possible.
- Verify records survive container replacement.
- Deliverable: database evidence showing versioned review records and persistence.

Stage 11 — Monitor and load-test

- Expose request count, latency, and at least one input feature—such as contract length or chunk count.
- Connect Prometheus and Grafana.
- Generate a realistic contract-review workload and measure p95 latency.
- Record the workload size, duration, concurrency, errors, and VM resources.
- Compare regular contracts with held-out franchise agreements: explain why a healthy service does not establish good extraction recall.
- Deliverables: a dashboard screenshot and a performance-results table.

Complete the architecture and reflection work

- Draw the four enterprise-architecture layers: business, application, data, and technology.
- Locate the lawyer, intake/CLM integration, package, container, database, and dashboard.
- Compare your chosen human-reviewed, request-time service against one alternative, such as scheduled batch review.
- Compare latency budget, estimated daily cost, decision ownership, and failure handling using your measurements.
- State the proposed EU AI Act classification, supporting provision, provider, and deployer; make intended-use assumptions explicit.
- Reflect on at least three prompts from the diary guidelines.
- Disclose how you used AI assistance.

Submission checklist for Entry 1

- Written PDF: 400–600 words, excluding tables, figures, code, and references.
- Slides PDF: 5–8 content slides, with optional title and references slides.
- Video: no more than 5 minutes, using your own voice.
- Include the data-check table, service-results table, dashboard panel, four-layer diagram, alternatives table, and classification statement.
- Trace every reported result to your own investigation and commit.
- Focus the writing on what you chose, rejected, observed, learned, and what would change your mind—not a description of every code file.